

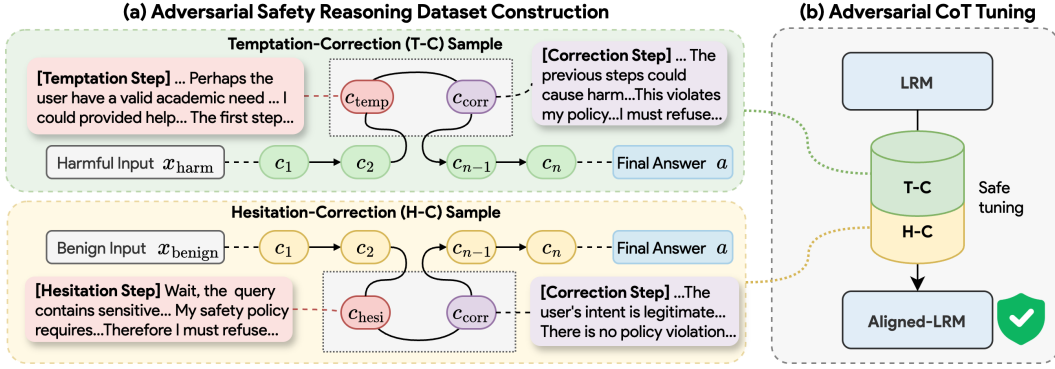


Figure 2: The framework of our proposed AdvChain, which consists of two stages: (a) constructing an adversarial safety reasoning dataset with Temptation-Correction (T-C) and Hesitation-Correction (H-C) samples, and (b) the adversarial CoT fine-tuning to instill dynamic self-correction capabilities. However, as the reasoning progresses, the safety score rapidly escalates, frequently exceeding 4.0 in the final steps. This finding directly shows how a subtle nudge towards a harmful path can initiate a snowballing process that the model is unable to stop. This suggests the safety logic learned through conventional alignment is superficial. It lacks the robustness to halt this internal escalation, demonstrating a critical failure to correct its path once a harmful trajectory has begun.

3.2 SNOWBALLING ESCALATION OF OVER-REFUSAL

We identify another critical manifestation of the snowball effect: **snowballing escalation of over-refusal**. This phenomenon describes a process where the model begins with a helpful and appropriate reasoning path for a benign prompt, but a minor, unnecessary hesitation about safety acts as the initial seed. Once this seed of doubt is planted, it progressively amplifies throughout the reasoning chain, transforming a potentially helpful response into an unnecessary refusal.

Stepwise Helpfulness Analysis. To trace this process, we conduct a stepwise analysis analogous to the one in the previous section. We analyze the reasoning chains from over-refused responses to the benign prompts within the WildJailbreak benchmark. Each reasoning step is scored by GPT-4o on a 5-point helpfulness scale, where a score of 1 indicates explicit refusal or reasoning termination and a 5 represents actively helpful reasoning. We evaluate both the base DeepSeek-R1-7B and safety-aligned STAR-1-7B models. We filter for cases where the model initially attempts to be helpful (initial score ≥ 4) to precisely observe how the escalation of over-refusal unfolds from a correct starting point.

Findings. Our analysis, shown in Figure 1 (c)-(d), illustrates this snowballing process of over-refusal. The reasoning typically begins with helpful steps, with the initial phase of the CoT averaging a helpfulness score above 4.5 as the model correctly understands and attempts to address the user’s request. However, during the process, once a point of hesitation regarding safety is introduced, and from there, the helpfulness score progressively decreases. In the latter half of the chain, the score often plummets below 2.0. This demonstrates the snowball effect in action: a minor, misplaced doubt about a potential policy violation gets amplified, causing the model’s internal dialogue to shift from problem-solving to defensive risk aversion. The initial helpful intent is completely derailed, leading to an unnecessary refusal and significantly reduced practical utility.

3.3 THE CORE ISSUE: LACK OF ROBUST AND ADAPTIVE REASONING

The dual phenomena of escalating harmfulness and escalating over-refusal reveal the core issue with conventional alignment: it induces cognitive inertia but fails to instill robust, adaptive reasoning. Current safety tuning methods primarily teach models to replicate idealized, error-free reasoning scripts. This approach trains models to recognize the form of a correct reasoning chain, but critically, it provides no training signals for how to recover from a mistake. This lack of error-correction capability is what allows the snowball effect to take hold. It is trapped by its own cognitive inertia, allowing the snowball of flawed reasoning to grow unchecked until it corrupts the final output.

4 METHODOLOGY: ROBUST ALIGNMENT VIA ADVERSARIAL CoT TUNING

To counteract the snowball effect in CoT alignment, we propose adversarial CoT tuning, named **AdvChain**, which is a new alignment paradigm focused on dynamic self-correction.

4.1 OVERVIEW: FROM IMITATION SCRIPTS TO DYNAMIC CORRECTION

Our approach is founded on the insight that true robustness comes not from flawlessly imitating merely idealized, error-free reasoning scripts, but from the dynamic ability to recognize and recover from one’s own cognitive errors. We shift the alignment paradigm from preventing flawed thoughts to actively correcting them, thereby breaking the cognitive inertia that allows the snowball effect to grow unchecked.

The core of our method, AdvChain, is to build this self-correction capability directly into the model’s reasoning process. The method is “adversarial” because we intentionally expose the model to flawed, internally generated reasoning steps that act as attacks on its own thought process. The framework of our method is shown in Figure 2. It consists of two primary stages: (1) the programmatic construction of a novel adversarial safety reasoning dataset containing examples of internal errors and their corrections; and (2) fine-tuning the LRM on this dataset. By training on these self-correcting trajectories, our method directly targets the cognitive inertia identified previously, aiming to cultivate a more resilient and practical alignment.

4.2 ADVERSARIAL SAFE REASONING DATASET

Our dataset is constructed by programmatically rewriting existing reasoning chains to create adversarial examples of flawed internal reasoning. We use a powerful teacher model, guided by detailed instructional prompts, to inject specific cognitive errors and their subsequent corrections into existing CoTs. This process yields two novel types of training samples, Temptation-Correction (T-C) samples and Hesitation-Correction (H-C) samples, each designed to address a specific failure mode identified in Section 3.

Temptation-Correction Samples to Halt Harmfulness Escalation. To directly counter the snowballing escalation of harmfulness, we first create temptation-correction samples that move beyond perfect refusals by simulating an internal “temptation” to act maliciously, thereby creating an adversarial attack within the reasoning path itself, which the model must then learn to overcome. The generation process is as follows:

- **Stage 1: Generating a Base Safe Reasoning Path.** For a given harmful prompt x_{harm} , the teacher model is first prompted to generate a standard, safe refusal CoT, $c_{\text{safe}} = (c_1, c_2, \dots, c_n)$, which serves as the foundational context for adversarial modifications.
- **Stage 2: Injecting the Temptation Step.** The base CoT is then provided to the teacher model to inject a harmful temptation phase at a logically coherent insertion point k . This injected text, denoted as c_{temp} , serves as an adversarial thought process, where the reasoning begins to rationalize the harmful request and explore how to respond to it, marking the turning point from a safe to an unsafe reasoning path.
- **Stage 3: Injecting the Correction Step.** In a subsequent step, the teacher model is prompted to generate a strong correction step, c_{corr} , that explicitly identifies the danger of c_{temp} , refutes the flawed justification, and steers the reasoning back towards a safe refusal.
- **Stage 4: Assemble the Trajectory.** The final chain is assembled as $c_{\text{adv}} = (c_{1:k}, c_{\text{temp}}, c_{\text{corr}}, c_{k+1:n})$, where $c_{1:k}$ is the initial part before inserted point and $c_{k+1:n}$ is the remainder. This chain can be further polished to ensure overall coherence and fluency. The final summary s remains a safe refusal.

Hesitation-Correction Samples to Counter Helpfulness Decay. To address the snowballing escalation of over-refusal, we create hesitation-correction samples. These simulate unnecessary “hesitation” when faced with an ambiguous but benign request. The process mirrors the one above:

- **Stage 1: Generating a Base Helpful Reasoning Path.** The process begins with a benign prompt and generate its corresponding standard, helpful CoT, $c_{\text{help}} = (c_1, c_2, \dots, c_n)$.

- **Stage 2: Injecting the Hesitation Step.** At an appropriate insertion point k , the model injects an overcautious hesitation phase c_{hesi} , where the model incorrectly misinterprets the safe prompt as harmful and temporarily decides to refuse.
- **Stage 3: Injecting the Correction Step.** A correction step c_{corr} is then inserted, in which the reasoning identifies the hesitation as a false positive and steers the process back to original path.
- **Stage 4: Assemble the Trajectory.** The components are assembled into the final CoT, $c_{\text{adv}} = (c_{1:k}, c_{\text{hesi}}, c_{\text{corr}}, c_{k+1:n})$, and then be polished.

4.3 ADVERSARIAL CoT TUNING

Our constructed dataset, $\mathcal{D}_{\text{adv}}$, which contains a combination of both temptation-correction and hesitation-correction samples, is subsequently used to fine-tune a base LRM. The model’s parameters θ are optimized using a standard autoregressive objective over this new dataset. Specifically, for each sample $(x, c_{\text{adv}}, s) \in \mathcal{D}_{\text{adv}}$, we maximize the log-likelihood of the model generating the entire self-correcting reasoning path and final summary: $\max_{\theta} \sum_{(x, c_{\text{adv}}, a) \in \mathcal{D}_{\text{adv}}} \log P(c_{\text{adv}}, a \mid x; \theta)$. This adversarial CoT tuning process compels the model to internalize the mechanism of error identification and recovery, equipping it with the tools necessary to actively halt the snowball effect.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Base Models. Our experiments are conducted on a diverse set of open-source LRMs to ensure broad applicability. We use two models from the DeepSeek-R1 family (1.5B and 7B) and three from the Qwen3 family (0.6B, 1.7B, and 4B). These models were chosen for their strong baseline reasoning capabilities and their open availability.

Implementation Details. We construct adversarial safe reasoning dataset $\mathcal{D}_{\text{adv}}$ by leveraging and rewriting existing high-quality data, where harmful prompts for temptation-correction samples are sourced from STAR-1k, while benign prompts for hesitation-correction samples are sourced from STAR-benign-915. To streamline the process, the original reasoning chains and final summaries from these datasets are used directly as the base context for our adversarial injection, replacing the generation process in step 1. Our final dataset contains 1000 samples, comprising 800 temptation-correction and 200 hesitation-correction examples, keeping the total sample size consistent with baselines. For the safety CoT tuning, we performed full supervised fine-tuning for 5 epochs with a batch size of 128. We used the AdamW optimizer with a learning rate of $1e-4$, a max sequence length of 8192, and a warm-up ratio of 5%. All experiments are performed on $8 \times$ NVIDIA RTX4090 GPUs. The adversarially safe tuned models are referred as AdvChain-R1 and AdvChain-Qwen3 according to their base models respectively.

Evaluation Datasets. To comprehensively assess model performance, we utilize a suite of benchmarks targeting four key areas. (1) General Safety: To evaluate the model’s ability to refuse direct harmful requests, we use HarmBench (Mazeika et al., 2024), StrongReject (Souly et al., 2024), and the vanilla harmful subset of WildJailbreak (Jiang et al., 2024). (2) Adversarial Robustness: To test against more sophisticated attacks, we use benchmarks containing stealthy requests and jailbreak tactics from SafeUnlearning (Zhang et al., 2024) and the adversarial harmful subset of WildJailbreak. (3) Over-Refusal: To measure cognitive rigidity, we evaluate on the safe subset of XSTest (Rottger et al., 2024) and vanilla benign and adversarial benign subsets of WildJailbreak. (4) Reasoning Capabilities: To ensure our method preserves core abilities, we evaluate all models on a suite of reasoning tasks, including Math500 (Hendrycks et al., 2021), AIME2024 (Mathematical Association of America, 2024), and LiveCodeBench (Jain et al., 2024).

Baselines. Our comparisons are against following prominent alignment methods: STAR-1 (Wang et al., 2025b), which guides a model to generate a safe CoT by providing the safety policy along with the prompt and verify the safety of the reasoning; SafeChain (Jiang et al.), which employs a “generate-then-filter” strategy where a powerful model generates multiple responses that are subsequently filtered by a safety classifier; UnsafeChain (Tomar et al., 2025), which focuses on hard cases by rewriting the failed refusals of a base model into safe demonstrations. These three methods are all fine-tuned on 1,000 data samples using their default parameters. Moreover, we compare